



Department of Computing And Information System

Object Oriented Programing Lab Report-2

Lab Report Name:

Java Basics

**Course Name: Object Oriented
Programing Lab**

Course Code: CIS216L

Submitted By:

Syed Rahimeen Zaman

Reg: 253-16-009

Semester: Fall 2026

Submitted To:

Mr. Md. Nasimul Kader

Assistant Professor

Department of Computing and Information System

Date of Submission: 15.08.2026

Java Basics:

*Hello world code

*Comments Example

*Data type demo

*Variable demo

*Simple operator demo

*Decision making demo

*loops

Public Github repository of Lab report 2:

<https://github.com/rahiminzaman/Object-Oriented-Programing-Lab-Report-2>

Hello world code

Source Code:

```
public class HelloWorld {  
    public static void main(String[] args)  
    {  
        System.out.println("Hello, World");  
    }  
}
```

Output:

```
"C:\Program Files\Java\jdk-26.0.2.  
Hello, World  
  
Process finished with exit code 0
```

Comments Example

Source Code:

```
// We use this for single line comment which helps us understand the code
better and these lines are ignored by the JAVA compiler

/*
  For
  multi-line
  comments
*/
public class CommentsExample { // We define a class named Experiment1
(this has to match exactly with our original file name)
    public static void main(String[] args) {

        System.out.println("We will write a single-line comment beside this
code and that will be ignored by the compiler"); // This just prints our
lines to the output
        System.out.println("We will write a multi-line comment beside this
code and that will be ignored by the compiler"); /* so this is basically
multi-lines
        comments we can go as long as we like but now we will close it so
it doesn't affect the rest of the code */
        System.out.println("program executed successfully");

    }
}
```

Output:

```
// We use this for single line comment which helps us understand the code better and these lines are ignored by the JAVA compiler

/*
For
multi-line
comments
*/

public class CommentsExample { // We define a class named Experiment1 (this has to match exactly with our original file name)
    public static void main(String[] args) {

        System.out.println("We will write a single-line comment beside this code and that will be ignored by the compiler"); // This just prints our lines to the output
        System.out.println("We will write a multi-line comment beside this code and that will be ignored by the compiler"); /* so this is basically multi-lines
        comments we can go as long as we like but now we will close it so it doesn't affect the rest of the code */
        System.out.println("program executed successfully");
    }
}
```

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe" "-javaagent:D:\New folder (6)\IntelliJ IDEA 2024.2.1\lib\idea_rt.jar=12174:D:\New folder (6)" -Dfile.encoding=UTF-8
We will write a single-line comment beside this code and that will be ignored by the compiler
We will write a multi-line comment beside this code and that will be ignored by the compiler
program executed successfully

Process finished with exit code 0
```

Data type demo

Source Code:

```
public class DataTypesDemo {
    public static void main(String[] args) {

        // =====
        // 1. PRIMITIVE DATA TYPES (Basic values)
```

```

// =====

// Integer numbers
byte age = 20;           // 1 byte  (-128 to 127)
short distance = 1500;   // 2 bytes  (-32,768 to 32,767)
int salary = 50000;      // 4 bytes  (Standard number type)
long population = 8000000000L; // 8 bytes  (Large numbers, needs 'L'
at the end)

// Decimal numbers
float height = 5.9f;     // 4 bytes  (Needs 'f' at the end)
double gpa = 3.85;       // 8 bytes  (Standard decimal type)

// Single character & True/False
char grade = 'A';        // 2 bytes  (Stores a single character
in single quotes)
boolean isPassed = true; // 1 bit    (Stores true or false)

// =====
// 2. NON-PRIMITIVE DATA TYPES (Objects/Arrays)
// =====

String studentName = "Alex"; // Text sequence in double
quotes
int[] testScores = {90, 85, 88}; // Array storing multiple
values

// =====
// 3. DISPLAYING OUTPUTS
// =====

System.out.println("--- Student Details ---");
System.out.println("Name: " + studentName);
System.out.println("Age: " + age + " years");
System.out.println("Height: " + height + " ft");
System.out.println("GPA: " + gpa);
System.out.println("Grade: " + grade);
System.out.println("Passed Course: " + isPassed);

System.out.println("\n--- Additional Info ---");
System.out.println("Monthly Salary: $" + salary);
System.out.println("City Population: " + population);

// Printing array elements using a simple loop
System.out.print("Test Scores: ");
for (int score : testScores) {
    System.out.print(score + " ");
}
System.out.println(); // Prints a new line at the end
}
}

```

Output:

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"  
--- Student Details ---  
Name: Alex  
Age: 20 years  
Height: 5.9 ft  
GPA: 3.85  
Grade: A  
Passed Course: true  
  
--- Additional Info ---  
Monthly Salary: $50000  
City Population: 8000000000  
Test Scores: 90 85 88  
  
Process finished with exit code 0
```

Variable demo

Source Code:

```
public class VariablesDemo {

    // 1. INSTANCE VARIABLE
    // Lives inside the class, belongs to each object/instance
separately
    int studentAge = 20;

    // 2. STATIC VARIABLE
    // Shared across ALL objects (only one copy exists in memory)
    static String schoolName = "Tech University";

    // Method to demonstrate variables
    public void displayDetails() {

        // 3. LOCAL VARIABLE
        // Declared inside a method; destroyed when method finishes
        int score = 95;

        // 4. FINAL VARIABLE
        // Constant value; cannot be modified once set
        final double MIN_PASSING_GRADE = 3.0;

        System.out.println("--- Student Record ---");
        System.out.println("Student Age (Instance): " + studentAge);
        System.out.println("School (Static): " + schoolName);
        System.out.println("Exam Score (Local): " + score);
        System.out.println("Passing Grade (Final): " +
MIN_PASSING_GRADE);
    }

    public static void main(String[] args) {

        // Creating an object to access instance variables and
methods
        VariablesDemo student1 = new VariablesDemo();
        student1.displayDetails();

        System.out.println("\n--- Static Access ---");
        // Static variables can be accessed directly using the class
name
        System.out.println("School Name via Class: " +
VariablesDemo.schoolName);
    }
}
```

```
}
```

Output:

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\jav
--- Student Record ---
Student Age (Instance): 20
School (Static): Tech University
Exam Score (Local): 95
Passing Grade (Final): 3.0

--- Static Access ---
School Name via Class: Tech University

Process finished with exit code 0
```

Simple operator demo

Source Code:

```
public class SimpleOperatorsDemo {
    public static void main(String[] args) {
        int a = 10, b = 3;

        // Arithmetic Operators
        System.out.println("a + b = " + (a + b)); // Addition
        System.out.println("a - b = " + (a - b)); // Subtraction

        // Relational Operator
        System.out.println("a > b ? " + (a > b)); // Greater than

        // Logical Operator
        boolean x = true, y = false;
        System.out.println("x && y = " + (x && y)); // Logical AND

        // Assignment Operator
        a += 5; // a = a + 5
        System.out.println("a after += 5 : " + a);

        // Ternary Operator
        int max = (a > b) ? a : b;
        System.out.println("Maximum = " + max);
    }
}
```

Output:

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\jav
a + b = 13
a - b = 7
a > b ? true
x && y = false
a after += 5 : 15
Maximum = 15

Process finished with exit code 0
```

Decision making demo

Source Code:

```
public class DecisionMakingDemo {  
    public static void main(String[] args) {  
        int number = 10;  
  
        // 1. SIMPLE IF STATEMENT  
        // Checks if a single condition is true  
        if (number > 0) {  
            System.out.println("The number is positive.");  
        }  
  
        // 2. IF-ELSE STATEMENT  
        // Chooses between two paths (even or odd)  
        if (number % 2 == 0) {  
            System.out.println("The number is even.");  
        } else {  
            System.out.println("The number is odd.");  
        }  
    }  
}
```

```

// 3. IF-ELSE-IF LADDER
// Checks multiple conditions in sequence
if (number < 0) {
    System.out.println("The number is negative.");
} else if (number == 0) {
    System.out.println("The number is zero.");
} else {
    System.out.println("The number is positive.");
}

// 4. SWITCH STATEMENT
// Selects a code block based on a single exact value
int day = 3;
switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    case 3:
        System.out.println("Wednesday");
        break;
    default:
        System.out.println("Other day");
        break;
}
}
}

```

Output:

```

C:\Program Files\Java\jdk-26.0.2.1\bin
The number is positive.
The number is even.
The number is positive.
Wednesday

Process finished with exit code 0

```

Loops Demo

Source Code:

```
public class LoopsDemo {
    public static void main(String[] args) {

        // 1. FOR LOOP
        // Used when you know exact number of iterations in advance
        System.out.println("--- 1. For Loop ---");
        for (int i = 1; i <= 5; i++) {
            System.out.println("Count: " + i);
        }

        // 2. WHILE LOOP
        // Checks condition FIRST, then executes as long as condition is
true        System.out.println("\n--- 2. While Loop ---");
        int j = 1;
        while (j <= 5) {
            System.out.println("Count: " + j);
            j++; // Increment counter so loop doesn't run forever
        }

        // 3. DO-WHILE LOOP
        // Executes block FIRST at least once, then checks condition
        System.out.println("\n--- 3. Do-While Loop ---");
        int k = 1;
        do {
            System.out.println("Count: " + k);
            k++; // Increment counter
        } while (k <= 5);

        // 4. ENHANCED FOR LOOP (FOR-EACH LOOP)
        // Used to iterate through items in an array without using indexes
        System.out.println("\n--- 4. Enhanced For Loop ---");
        int[] scores = {10, 20, 30, 40, 50};
        for (int score : scores) {
            System.out.println("Score: " + score);
        }
    }
}
```

Output:

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
--- 1. For Loop ---
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5

--- 2. While Loop ---
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5

--- 3. Do-While Loop ---
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5

--- 4. Enhanced For Loop ---
Score: 10
Score: 20
Score: 30
Score: 40
Score: 50

Process finished with exit code 0
```

Public Github repository of Lab report 2:

<https://github.com/rahiminzaman/Object-Oriented-Programing-Lab-Report-2>